



DEPARTAMENTO DE INFORMÁTICA
CURSO 2011-2012

Ingeniería Telecomunicaciones -Sistemas Electrónicos
Redes de Transmisión de Datos

Tema 6

Niveles de red y transporte

Prof. Juan Manuel Orduña Huertas

Contenidos

1	La capa de red	1
1.1	El datagrama IP	1
1.2	Direccionamiento IP	7
1.2.1	Subredes	10
1.2.2	Superredes: Classless InterDomain Routing (CIDR)	15
1.3	Protocolos de control de Internet	16
1.3.1	ICMP (Internet Control Message Protocol)	16
1.3.2	ARP (Address Resolution Protocol)	18
1.3.3	RARP, BOOTP y DHCP	19
2	El nivel de Transporte	22
2.1	Primitivas del servicio de transporte	23
2.2	La interfaz <i>sockets</i>	24
2.3	Los protocolos de transporte de la Internet: TCP y UDP	25
2.3.1	TCP (Transport Control Protocol)	25
2.3.2	La cabecera de segmento TCP	28

2.4	UDP (User Datagram Protocol)	30
	Bibliografía	34

1 LA CAPA DE RED

La Internet es un compendio de redes diferentes que comparten una pila de protocolos comunes. Cada una de estas redes es administrada por una entidad diferente: universidades, redes académicas nacionales, proveedores comerciales también llamados ISPs (Internet Service Providers), operadores, empresas multinacionales, etc. Como consecuencia de esto las políticas de uso son muy variadas.

Técnicamente, a nivel de red la Internet puede definirse como un conjunto de redes conectadas entre sí que utilizan el protocolo de red IP. A su vez, una red IP es una red que utiliza datagramas, no orientada a conexión y con servicio best effort (es decir, no ofrece calidad de servicio o QoS (Quality of Service)). La entrega de los paquetes no está garantizada, ya que en momentos de congestión éstos pueden ser descartados sin previo aviso por los routers que se encuentren en el trayecto.

1.1 El datagrama IP

En una red IP absolutamente toda la información viaja en datagramas IP. Esto incluye tanto la intercambiada a nivel de transporte por TCP y UDP como cualquier información de control que tenga que intercambiarse, por ejemplo para routing dinámico, mensajes de error, etc.

El datagrama IP tiene dos partes: cabecera y texto; la cabecera tiene una parte fija de 20 bytes y una opcional de entre 0 y 40 bytes. La longitud total de la cabecera siempre es múltiplo de 4; esto garantiza un proceso eficiente por parte de equipos (hosts o routers), cuya arquitectura optimiza el acceso a direcciones de memoria que estén en frontera de 32 bits. La estructura de la cabecera IP es la que se muestra en la tabla 1.1

El campo **versión** permite que coexistan en la misma red sin ambigüedad paquetes de distintas versiones de IP; la versión actualmente utilizada (que corresponde a la estructura de datagrama que estamos viendo) es la 4. Como veremos mas tarde, se empieza a extender el uso de una nueva versión (la 6) con una estructura de datagrama diferente.

<i>Campo</i>	<i>Longitud (bits)</i>
Versión	4
IHL (Internet Header Length)	4
DS (Differentiated Services)	8
Longitud total	16
Identificación	16
Reservado	1
DF (Don't Fragment)	1
MF (More Fragments)	1
Fragment offset	13
TTL (Time To Live)	8
Protocolo (de transporte)	8
Checksum (de cabecera)	16
Dirección de origen	32
Dirección de destino	32
Opciones	0-320

Tabla 1.1: Estructura de la cabecera IP

El campo **IHL** especifica la longitud de la cabecera, ya que ésta puede variar debido a la presencia de campos opcionales. Se especifica en palabras de 32 bits. La longitud mínima es 5 y la máxima 15, que equivale a 40 bytes de información opcional. La longitud de la cabecera siempre ha de ser un número entero de palabras de 32 bits, por lo que si la longitud de los campos opcionales no es un múltiplo exacto de 32 bits se añade un relleno al final de la cabecera.

El campo **Differentiated Services** ha sustituido recientemente al antes denominado tipo de servicio. Su finalidad es implementar Calidad de Servicio en redes IP mediante la arquitectura denominada Servicios Diferenciados o Diffserv. La descripción de este campo y su funcionamiento la haremos más adelante al hablar de Calidad de Servicio.

El campo **longitud total** especifica la longitud del datagrama completo (cabecera incluida) en bytes. El valor máximo es 65535 bytes. Este campo sirve para saber donde termina el datagrama información que es realmente necesaria sólo en muy pocos casos, ya que en la mayoría de situaciones puede deducirse a partir de la información que posee el nivel de enlace. Por ejemplo en PPP, ATM o LANs no Ethernet el nivel de enlace sabe o puede deducir fácilmente la longitud del datagrama; el único caso en Ethernet donde es preciso indicar la longitud del datagrama es cuando se utiliza encapsulado DIX (donde el campo longitud se ha reemplazado por el campo Ethertype) y se envía una trama que no llega a 64 bytes; en este caso la trama contendrá relleno y la única forma de averiguar donde empieza éste es a partir de la longitud total indicada en la cabecera IP.

Los cuatro campos siguientes (Identificación, DF, MF y Fragment Offset) están relacionados con la fragmentación de datagramas que explicaremos más adelante.

El campo **TTL (Time To Live)** sirve para descartar un datagrama cuando ha dado un número excesivo de saltos o ha pasado un tiempo excesivo viajando por la red y es presumiblemente inútil. Se trata de un contador regresivo que indica el tiempo de vida restante del datagrama medido en segundos, de forma que si llega a valer cero el datagrama debe ser descartado. Además cada router por el que pasa dicho datagrama está obligado a restar uno del TTL, independientemente del tiempo que tarde en reenviarlo. Esto evita que por algún problema en las rutas se produzcan bucles y un datagrama permanezca 'flotando' indefinidamente en la red. Para medir el tiempo que un datagrama emplea en pasar de un router a otro sería preciso que los relojes de ambos estuvieran sincronizados; como esto no siempre ocurre en la práctica el tiempo en tránsito se ignora y el TTL se convierte simplemente en un contador de saltos; en muchas implementaciones ni siquiera cuando el datagrama esta mas de un segundo esperando en un router se disminuye por ello el valor del TTL, aunque esto ocurre raramente. En el caso particular de producirse fragmentación el host receptor sí puede retener datagramas durante varios segundos, mientras espera a recibir todos los fragmentos; en este caso el host sí disminuirá el valor del TTL en uno por cada segundo de espera, pudiendo llegar a descartar datagramas por este motivo, como veremos más adelante. Es habitual utilizar para el TTL los valores 64 o 255. El comando de prueba ping permite fijar el valor inicial del TTL en los datagramas de prueba enviados; por ejemplo en los sistemas operativos windows se utiliza para este fin la opción `-i` del comando ping.

El campo protocolo especifica a que protocolo del nivel de transporte corresponde el datagrama. La tabla de protocolos válidos y sus correspondientes números son controlados por el IANA (Internet Assigned Number Authority) y pueden consultarse en su web, www.iana.org/numbers.html. La tabla 1.2 muestra a título de ejemplo algunos de los posibles valores de este campo.

Merece la pena llamar la atención sobre el valor 4 del campo protocolo, que está reservado para el uso de IP para transportar IP, es decir al encapsulado de un datagrama IP dentro de otro. Esta técnica permite realizar un encaminamiento desde el origen de los paquetes encapsulando el datagrama en otro dirigido al nodo intermedio por el que queremos pasar. Esto es como si para enviar una carta a Sevilla vía Madrid metiéramos el sobre dirigido a Sevilla en otro que lleve una dirección de Madrid; la carta llegaría a Madrid y de allí se enviaría a Sevilla.

El campo checksum sirve para detectar errores producidos en la cabecera del datagrama; no es un CRC sino el complemento a uno en 16 bits de la suma complemento a uno de toda la cabecera tomada en campos de 16 bits (incluidos los campos opcionales si los hay). Para el cálculo el campo checksum se pone a sí mismo a ceros. El checksum permite salvaguardar al datagrama de una alteración en alguno de los campos de la cabecera que pudiera producirse por ejemplo por un problema hardware en un router. Obsérvese que el checksum solo cubre la cabecera del datagrama, no los datos. El campo checksum se ha de recalcular en cada salto, ya que al menos el TTL cambia. En routers con mucho tráfico el recálculo del checksum supone un inconveniente desde el punto de vista de rendimiento.

Valor	Nombre	Descripción
0		Reservado
1	ICMP	Internet Control Message Protocol
2	IGMP	Internet Group Management Protocol
3	GGP	Gateway-to-Gateway Protocol
4	IP	IP en IP (encapsulado)
5	ST	Stream
6	TCP	Transmission Control Protocol
8	EGP	Exterior Gateway Protocol
17	UDP	User Datagram Protocol
29	ISO-TP4	ISO Transport Protocol Clase 4
38	IDRP-CMTP	IDRP Control Message Transport Protocol
80	ISO-IP	ISO Internet Protocol (CLNP)
88	IGRP	Interior Gateway Routing Protocol (Cisco)
89	OSPF	Open Shortest Path First
255		Reservado

Tabla 1.2: Ejemplos de algunos protocolos que pueden codificarse en la cabecera de un datagrama IP

Los campos dirección de origen y dirección de destino corresponden a direcciones IP de cuatro bytes según el formato que veremos luego.

Los campos opcionales de la cabecera no siempre están soportados en los routers y se utilizan muy raramente. Cada campo opcional está compuesto por una etiqueta, seguida opcionalmente de información adicional. Los más importantes son los siguientes:

Record route (registrar la ruta): Esta opción solicita a los routers por los que pasa que anoten su dirección en la cabecera del datagrama, con lo que al final del trayecto se dispone de una traza de la ruta seguida para fines de prueba o diagnóstico de problemas. Esto es como si cada router estampara su sello en el datagrama antes de reenviarlo. El máximo de direcciones que puede registrarse es de 9, ya que si se registraran 10 se ocuparían los 40 bytes de opciones y no quedaría sitio para la propia opción record route. El comando de prueba ping permite solicitar el uso de este campo para registrar la ruta; por ejemplo en los sistemas operativos windows se utiliza para este fin la opción ?r del comando ping.

Timestamp (marca de tiempos): esta opción actúa de manera similar a record route, pero el router en lugar de anotar su dirección anota el instante (tiempo desde la medianoche en milisegundos) en el que el datagrama pasa por él; es un campo entero de 32 bits. También puede acompañarse el timestamp de la dirección IP del router; en este caso cada anotación ocupa ocho bytes y solo pueden producirse cuatro ya que la quinta superaría el tamaño máximo del campo opciones (la propia indicación de que se trata de un campo timestamp ya ocupa cuatro bytes). El comando

de prueba ping permite solicitar el uso de este campo para registrar la ruta y hora; por ejemplo en los sistemas operativos windows se utiliza para este fin la opción -s del comando ping.

Source routing (encaminamiento desde el origen): permite al emisor especificar la ruta que debe seguir el datagrama hasta llegar a su destino. Existen dos variantes: strict source routing (encaminamiento estricto desde el origen) especifica la ruta exacta salto a salto, de modo que si en algún caso la ruta marcada no es factible se producirá un error (una ruta no es factible cuando el router siguiente en la secuencia no es accesible directamente, es decir a nivel de enlace, desde el router actual). El funcionamiento de esta opción es equivalente al de los puentes con encaminamiento desde el origen. La opción loose source routing (encaminamiento aproximado desde el origen) indica una serie de routers por los que debe pasar un datagrama, pero deja la posibilidad de pasar por otros no especificados para llegar al siguiente router marcado en la secuencia. La limitación en la longitud de las opciones impone también aquí un límite máximo en el número de saltos que pueden especificarse en estas opciones, por lo que a menudo se utiliza en su lugar la técnica antes descrita de encapsulado IP en IP. Ambas variantes actúan también como record route, pues la secuencia no se va borrando a medida que se utiliza con lo que al final del trayecto el campo opciones contiene la ruta que se ha utilizado. El comando de prueba ping permite solicitar el uso de estos campos para especificar la ruta; por ejemplo en los sistemas operativos windows se utilizan las opciones ?j y ?k para indicar loose source routing y strict source routing, respectivamente.

El tamaño de un datagrama IP se especifica en un campo de dos bytes en la cabecera, por lo que su valor máximo es de 65535 bytes, pero muy pocos protocolos o tecnologías a nivel de enlace admiten enviar tramas de semejante tamaño. Normalmente el nivel de enlace no fragmenta¹, por lo que tendrá que ser IP el que adapte el tamaño de los datagramas para que quepan en las tramas del nivel de enlace; por tanto en la práctica el tamaño máximo del datagrama viene determinado por el tamaño máximo de trama característico de la red utilizada. Este tamaño máximo de datagrama se conoce como MTU (Maximum Transfer Unit).

Podemos distinguir dos situaciones en las que se produce fragmentación. La primera, que podemos denominar fragmentación en ruta, se produce cuando un datagrama es creado por un host en una red con un valor determinado de MTU y en su camino hacia el host de destino ha de pasar por otra red con una MTU menor. Por ejemplo un datagrama creado en una Token Ring con MTU de 4440 bytes dirigido a una Ethernet con MTU de 1500 bytes. En estos casos el router que hace la transición a la red de MTU menor ha de fragmentar los datagramas para que no excedan el tamaño de la nueva red.

La segunda, que podemos llamar fragmentación en origen, se produce como consecuencia del diseño de la aplicación. Por ejemplo muchas implementaciones de NFS generan datagramas de 8 Kbytes de datos (8212 bytes con la cabecera IP). Un host en una red Ethernet que utilice NFS tendrá que fragmentar cada datagrama en seis fragmentos antes de enviarlo, aun cuando el host de

origen y destino se encuentren ambos en el mismo concentrador de Ethernet. Como podemos ver en la tabla 3.3 solo una minoría de las redes existente soporta datagramas NFS sin fragmentar.

La fragmentación se hace cortando la parte de datos del datagrama en trozos tales que cada fragmento con su cabecera quepa en la MTU de la nueva red (redondeado por abajo para que la cantidad de datos sea múltiplo de ocho bytes). Todos los campos de la cabecera del datagrama original se replican en los fragmentos excepto aquellos que se emplean para distinguir los fragmentos y que describiremos a continuación. Una vez fragmentado un datagrama no se reensambla hasta que llegue al host de destino, aun cuando en el trayecto pase a redes que admitan una MTU mayor. Los estándares Internet recomiendan que todas las redes que soporten TCP/IP tengan una MTU de al menos 576 bytes y esta condición la cumplen la mayoría de las redes, aunque no todas. La MTU mínima imprescindible para funcionar en TCP/IP es de 68 bytes, valor que corresponde a 60 bytes de cabecera (el máximo con todos los campos opcionales) y 8 bytes de datos, que es el fragmento mínimo de datos que puede hacerse.

El campo Identificación de la cabecera IP lo usa el emisor para marcar en origen cada datagrama emitido; de esta forma en caso de que se produzca fragmentación el receptor podrá reconocer las partes que corresponden al mismo datagrama original, ya que todas irán acompañadas de la misma identificación.

El bit DF (Don't Fragment) cuando está a 1 indica a los routers que este datagrama no debe fragmentarse.

El comando de prueba ping permite especificar la no fragmentación. Por ejemplo en los sistemas operativos windows se utiliza para este fin la opción ?f. Esto combinado con la especificación de un determinado tamaño de paquete (opción ?l en windows) permite hacer sondeos manuales de la MTU de un trayecto, lo cual resulta muy útil en la resolución de problemas o en pruebas de rendimiento.

El bit MF (More Fragments) puesto a 1 especifica que este datagrama es realmente un fragmento de un datagrama mayor, y que no es el último. Si está a 0 indica que este es el último fragmento o bien que el datagrama no ha sido fragmentado.

El campo Fragment offset sirve para indicar, en el caso de que el datagrama sea un fragmento, en que posición del datagrama original se sitúan los datos que contiene. Los cortes siempre se realizan en múltiplos de 8 bytes, que es la unidad elemental de fragmentación, por lo que el Fragment offset cuenta los bytes en grupos de 8. Gracias a eso este contador requiere únicamente 13 bits en vez de los 16 que harían falta si contara bytes ($2^{13}=8192$, $8192 \times 8 = 65536$). De los tres bits que se ganan dos se utilizan en los flags DF y MF y el tercero no se utiliza.

Los fragmentos de un datagrama pueden llegar desordenados a su destino; el receptor podrá identificarlos gracias al campo Identificación. La longitud total del datagrama puede calcularla

cuando recibe el último fragmento (identificado por el bit MF a 0) a partir de los campos Longitud y Fragment offset; la longitud será $\text{fragment_offset} * 8 + \text{longitud}$.

Cuando se fragmenta un datagrama el host receptor retiene en su buffer los fragmentos y los reensambla cuando los ha recibido todos. Mientras mantiene retenido un fragmento el host va restando cada segundo una unidad al campo TTL. Cuando el valor de TTL es igual a cero descarta el fragmento. Si alguno de los fragmentos de un datagrama se pierde el resto terminarán desapareciendo a medida que agoten su TTL. No existe ningún mecanismo en IP que contemple el reenvío de datagramas o de fragmentos. Si el protocolo utilizado a nivel superior contempla reenvío de paquetes perdidos (por ejemplo TCP a nivel de transporte) se provocará el reenvío del datagrama correspondiente. Normalmente el segundo envío se verá sometido a la misma fragmentación que el primero, pero el segundo no podrá en ningún caso aprovechar fragmentos residuales que pudiera haber en el host receptor correspondientes al primer envío, ya que desde el punto de vista del nivel IP se trata de dos datagramas distintos e independientes que reciben identificaciones diferentes.

1.2 Direccionamiento IP

Cada interfaz de red de cada nodo (host o router) en una red IP se identifica mediante al menos una dirección única de 32 bits. Las direcciones IP se suelen representar por cuatro números decimales separados por puntos, que equivalen al valor de cada uno de los cuatro bytes que componen la dirección. Por ejemplo una dirección IP válida sería 147.156.23.208.

Si un nodo dispone de varias interfaces físicas (cosa habitual en los routers) cada una de ellas deberá tener necesariamente una dirección IP distinta si se desea que sea accesible de forma diferenciada para este protocolo. Es posible también y en algunas situaciones resulta útil, definir varias direcciones IP asociadas a una misma interfaz física.

Como ocurre en la mayoría de las redes las direcciones IP tienen una estructura jerárquica. Una parte de la dirección corresponde a la red, y la otra al host dentro de la red. Cuando un router recibe un datagrama por una de sus interfaces compara la parte de red de la dirección con las entradas contenidas en sus tablas (que normalmente sólo contienen direcciones de red, no de host) y envía el datagrama por la interfaz correspondiente.

En el diseño inicial de la Internet se reservaron los ocho primeros bits para la red, dejando los 24 restantes para el host; se creía que con 254 redes habría suficiente para la red experimental de un proyecto de investigación del Departamento de Defensa americano. Pronto se vio que esto resultaba insuficiente, por lo que se reorganizó el espacio de direcciones reservando unos rangos para definir redes más pequeñas. El resultado de esa reorganización es lo que hoy conocemos como las redes clase A, B y C:

- Una red de clase A se caracteriza por tener a 0 el primer bit de dirección; el campo red ocupa los 7 bits siguientes y el campo host los últimos 24. Puede haber hasta 128 redes de clase A con 16777216 direcciones cada una.
- Una red de clase B tiene el primer bit a 1 y el segundo a 0; el campo red ocupa los 14 bits siguientes, y el campo host los 16 últimos. Puede haber 16384 redes clase B con 65536 direcciones cada una.
- Una red clase C tiene los primeros tres bits a 110; el campo red ocupa los siguientes 21 bits, y el campo host los 8 últimos. Puede haber hasta 2097152 redes clase C con 256 direcciones cada una.

Para indicar qué parte de la dirección corresponde a la red y qué parte al host se suele utilizar una notación denominada máscara, consistente en poner a 1 los bits que corresponden a la parte de red y a 0 los que corresponden a la parte host. Así por ejemplo diremos que una red clase A tiene una máscara 255.0.0.0, lo cual equivale a decir que los ocho primeros bits especifican la red y los 24 restantes el host. Análogamente decimos que una red clase B tiene una máscara 255.255.0.0 y una clase C una máscara 255.255.255.0.

Las máscaras permiten extraer de forma sencilla la parte de red o de host de una dirección. Por ejemplo un router que ha de enviar un datagrama puede realizar un AND entre la dirección de destino y la máscara correspondiente, con lo que extraerá la parte de red de la dirección.

Existen además direcciones (no redes) clase D cuyos primeros cuatro bits valen 1110. Las direcciones clase D se utilizan para definir grupos multicast. El grupo queda definido por los 28 bits siguientes. Puede haber hasta 268435456 direcciones multicast en Internet. Las direcciones clase D nunca puede aparecer como direcciones de origen de un datagrama.

Por último, la clase E, que corresponde al valor 1111 en los primeros cuatro bits, no se utiliza de momento y está reservada para usos futuros.

De los valores de los primeros bits de cada una de las clases antes mencionadas se puede deducir el rango de direcciones que corresponde a cada una de ellas. Así pues, en la práctica es fácil saber a que clase pertenece una dirección determinada sin más que observar el primer byte de su dirección. La tabla de la figura 1.1 resume la información esencial sobre las clases de redes de Internet.

La asignación de direcciones válidas de Internet la realizan los NICs (NIC = Network Information Center). Al principio había un NIC para toda la Internet pero luego se crearon NICs regionales. Actualmente existen los tres siguientes:

América: www.internic.net Europa: www.ripe.net Asia y Pacífico: www.apnic.net

Clase	Primero sbits	Bits red/host	Núm. Redes	Núm. Direcc.	Máscara	Rango de direcciones
A	0---	7/24	128	16777216	255.0.0.0	0.0.0.0-127.255.255.255
B	10--	14/16	16384	65536	255.255.0.0	128.0.0.0-191.255.255.255
C	110-	21/8	2097152	256	255.255.255.0	192.0.0.0-223.255.255.255
D	1110			268435456		224.0.0.0-239.255.255.255
E	1111					240.0.0.0-255.255.255.255

Figura 1.1: Clases de redes.

Estos NICs asignan direcciones IP a los proveedores de Internet y a las grandes organizaciones. Los proveedores a su vez asignan direcciones a las organizaciones de menor tamaño y a sus usuarios.

Existen unas reglas y convenios que asignan significados especiales a determinadas direcciones IP que es importante conocer:

1. La dirección 255.255.255.255 se utiliza para indicar broadcast en la propia red. La utilizaría por ejemplo como dirección de destino un host que está arrancando en una red local y que para averiguar la red en la que se encuentra y su propia dirección IP necesita localizar un servidor que le de los parámetros de configuración básicos. Solo se puede utilizar como dirección de destino, nunca como dirección de origen.
2. La dirección 0.0.0.0 identifica al host actual. La utilizaría el host del supuesto anterior como dirección de origen de sus datagramas. Solo se puede utilizar como dirección de origen, no de destino.
3. Las direcciones con el campo host todo a ceros identifican redes y por tanto no se utilizan para ningún host. Se emplean para especificar rutas y nunca deberían aparecer como direcciones de origen o destino de un datagrama. Por ejemplo la dirección 147.156.0.0 identifica la red clase B que pertenece a la Universidad de Valencia.
4. La dirección con el campo host todo a unos se utiliza como dirección broadcast dentro de la red y por tanto no se utiliza para ningún host. Solo puede ser una dirección de destino. Por ejemplo para enviar un mensaje broadcast a la red de la Universidad de Valencia utilizaríamos la dirección 147.156.255.255.
5. La dirección con el campo red todo a ceros identifica a un host en la propia red, cualquiera que esta sea; por ejemplo si queremos enviar un datagrama al primer host (0.1) de una red clase B podemos utilizar la dirección 0.0.0.1. Esto permite enviar un datagrama a un host en una red sin saber el número de ésta, aunque es preciso conocer si es clase A, B o C para saber que tamaño tiene la parte red de la dirección.

6. La dirección 127.0.0.1 se utiliza para pruebas loopback; todas las implementaciones de IP devuelven a la dirección de origen los datagramas enviados a esta dirección sin intentar enviarlos a ninguna parte¹.
7. Las redes 127.0.0.0, 128.0.0.0, 191.255.0.0, 192.0.0.0 y el rango de 240.0.0.0 en adelante (clase E) están reservados y no deben utilizarse.
8. Las redes 10.0.0.0 (clase A), 172.16.0.0 a 172.31.0.0 (clase B) y 192.168.0.0 a 192.168.255.0 (clase C) están reservadas para redes privadas ('intranets') por el RFC 1918; estos números no se asignan a ninguna dirección válida en Internet y por tanto pueden utilizarse para construir redes, por ejemplo detrás de un cortafuego, sin riesgo de entrar en conflicto de acceso con redes válidas de la Internet.

Como consecuencia de las reglas 3 y 4 antes mencionadas siempre hay dos direcciones inútiles en una red, la primera y la última. Por ejemplo, si tenemos la red 200.200.200.0 (clase C) tendremos que reservar la dirección 200.200.200.0 para denotar la red misma, y la dirección 200.200.200.255 para envíos broadcast a toda la red; dispondremos pues de 254 direcciones para hosts, no de 256.

1.2.1 Subredes

Supongamos que una empresa dispone de varias oficinas, cada una con una red local, todas ellas interconectadas entre sí, y que desea unir las mediante el protocolo TCP/IP; una de dichas oficinas (la principal) dispone además de una conexión a Internet. Supongamos también cada oficina tiene suficiente con 254 direcciones de hosts. En principio sería posible asignar una red clase C diferente para cada oficina, pero esto supone solicitar al NIC una red para cada oficina que se conecte, y al ser cada una independiente de las demás la gestión se complica; por ejemplo sería preciso anunciar en Internet la ruta para cada nueva red para que la oficina correspondiente fuera accesible. Dado que cada red sería en principio independiente de las demás no habría una forma sencilla de agrupar las redes de la organización.

Hay un mecanismo que permite dividir una red IP en trozos o subredes, de forma que la empresa de nuestro ejemplo podría solicitar una clase B y asignar fragmentos de dicha red a cada oficina a medida que se fueran incorporando a la red. Esto equivale a crear un nivel jerárquico intermedio entre la red y el host, permitiendo así grados variables de agrupación según el nivel en el que nos encontramos. Supongamos que a la empresa de nuestro ejemplo se le asigna una red clase B, la 156.134.0.0; de los 16 bits que en principio corresponden al host podría reservar los primeros 8 para la subred y dejar los 8 siguientes para el host, con lo que dispondrá de 256 subredes

¹Según el estándar todas las direcciones de la red 127.0.0.0 deberían ser loopback, pero en muchos casos esto sólo funciona para la primera dirección (127.0.0.1).

de 256 direcciones cada una. Desde fuera la red de la empresa seguirá siendo 156.134.0.0, ya que la estructura de subred no será visible.

Las subredes se añadieron a la Internet en 1982; con ello se consiguió una mayor flexibilidad en el reparto de direcciones dentro de una red.

Para dividir la red en subredes se define una nueva máscara. Como siempre los bits a 1 de la máscara identifican la parte de red (en este caso la parte de red-subred) y los bits a cero corresponden al host. Por ejemplo, la máscara 255.255.255.0 aplicada sobre una red clase B la divide en 256 subredes de 256 direcciones cada una, pues tiene puestos a 1 los primeros 24 bits; en cierto modo podríamos decir que esta máscara convierte una red clase B en 256 subredes clase C. Se pueden hacer divisiones que no correspondan a bytes enteros, por ejemplo la máscara 255.255.252.0 hace subredes mas grandes, reserva los primeros 6 bits para la subred y deja 10 para el host, con lo que podría haber hasta 64 subredes con 1024 direcciones cada una.

Cuando se crean subredes hay dos direcciones en cada subred que quedan automáticamente reservadas: las que corresponden al campo host todo a ceros y todo a unos; estas se emplean para designar la subred y para broadcast dentro de la subred, respectivamente. Así si la red 156.134.0.0 se subdivide con la máscara 255.255.255.0 se crean 256 subredes del tipo 156.134.subred.host, cada una con 256 direcciones. En cada subred hay 254 direcciones aprovechables para hosts, ya que la primera dirección (156.134.subred.0) identifica a la subred y la última (156.134.subred.255) es la dirección broadcast de esa subred. Por tanto el número de hosts de una subred es siempre dos menos que el rango de direcciones que abarca. Una consecuencia de lo anterior es que resulta absurdo crear subredes con la máscara 255.255.255.254, ya que el campo host tendría un bit solamente y no quedaría ninguna dirección aprovechable para hosts.

Del mismo modo que los valores todo ceros o todo unos del campo host están reservados con un significado especial, los valores todo ceros y todo unos del campo subred (es decir la primera y la última subredes) también son especiales. El valor todo ceros se utiliza para representar la subred misma; por ejemplo si a la red 156.134.0.0 le aplicamos la máscara 255.255.255.0 la primera subred (campo subred todo a ceros) no debería utilizarse, pues resultaría ambiguo el significado de la dirección 156.134.0.0, que representaría tanto a dicha subred como a la red entera. Análogamente la última subred (campo subred todo a unos) tampoco debería utilizarse porque entonces la dirección 156.134.255.255 significaría tanto broadcast en dicha subred como en la red entera. Por consiguiente en el campo subred también se pierden siempre dos direcciones, y tampoco tendría sentido crear máscaras con el campo subred de un bit, como sería el caso de una máscara 255.255.128.0 en el caso de una red clase B.

Mientras que la restricción de las direcciones todo ceros o todo unos en el campo host se ha de respetar siempre, existen muchas situaciones en las que interesa aprovechar la subred toda a ceros o toda a unos, violando la segunda norma antes mencionada. Esta violación, permitida por muchas implementaciones, se conoce como subnet-zero y se adopta para aprovechar mejor el espacio de

direcciones disponible; con subnet-zero es posible por ejemplo dividir una red clase B por la mitad en dos subredes mediante la máscara 255.255.128.0, cosa que no sería posible si no se permitiera esta pequeña 'infracción'.

En todos nuestros ejemplos la parte de subred de la máscara es contigua a la parte de red; en un principio se permitía crear subredes con máscaras no contiguas, por ejemplo dividir una clase B con la máscara 255.255.0.255; en este caso el host vendría especificado por el tercer byte y la subred por el cuarto. Dado que esta práctica solo resta claridad y hace menos eficiente el proceso de direcciones en los routers, actualmente la norma exige que la máscara sea contigua.

Para terminar de clarificar la creación de subredes y el uso de máscaras, resumimos en la tabla de la figura 1.2 todas las posibles subredes y máscaras que se pueden utilizar con una red clase B y con una red de clase C.

La división en subredes no ha de hacerse necesariamente de forma homogénea en todo el espacio de direcciones, como hemos hecho hasta ahora. Supongamos que la empresa de nuestro ejemplo anterior tiene una serie de oficinas pequeñas que tienen bastante con subredes de 256 direcciones, pero otras mas grandes requieren un número mayor; podríamos partir la red en 256 subredes de 256 hosts y asignar varias de esas subredes a las oficinas mayores, pero esto complicaría la gestión y las tablas de rutas. La solución más adecuada en este caso sería dividir la red en subredes de diferentes tamaños y asignar a cada oficina una subred adecuada a sus necesidades. Así en nuestro ejemplo se podría dividir la red 156.134.0.0 de forma que muestra la figura 1.3:

La técnica que hemos aplicado en el ejemplo anterior de dividir una red en subredes de diferentes tamaños se conoce comúnmente como máscaras de tamaño variable. Obsérvese que en este ejemplo hemos seguido la práctica de subnet-zero, es decir, se han considerado válidas subredes con el campo subred todo a ceros y todo a unos, de lo contrario no habría sido posible crear una subred con 32768 direcciones. Obsérvese en la columna de la derecha el uso de la notación subred/bits de máscara para representar las subredes; esta notación abreviada está sustituyendo en algunos casos a la tradicional y se emplea en la configuración de algunos equipos, en RFCs y otros documentos.

Cuando el campo subred ocupa bytes enteros es trivial la identificación de subredes. Las cosas se complican un poco más cuando no es así, por ejemplo en el caso anterior la primera subred de 1024 direcciones abarca un bloque de cuatro valores en el tercer byte, del 16 al 19; los bloques han de empezar en valores múltiplo de cuatro. Si en vez de empezar en el 16 hubiéramos empezado en el 15 (15, 16, 17 y 18) no habría sido posible crear una subred, pues la dirección 156.134.15.0 no tiene una máscara de 22 bits común con las otras tres; por consiguiente las subredes formadas con máscara de 22 bits (grupos de cuatro números) necesariamente han de empezar en valores múltiplos de cuatro (0, 4, 8, etc.). Análogamente puede deducirse que las subredes de máscara de 20 bits (4096 direcciones) han de empezar en valores múltiplos de 16, y así sucesivamente.

Bits de subred	Número de subredes	Nº subredes (subred-cero)	Bits de host	Número de hosts	Máscara
0	0	0	16	65534	255.255.0.0
1	0	2	15	32766	255.255.128.0
2	2	4	14	16382	255.255.192.0
3	6	8	13	8190	255.255.224.0
4	14	16	12	4094	255.255.240.0
5	30	32	11	2046	255.255.248.0
6	62	64	10	1022	255.255.252.0
7	126	128	9	510	255.255.254.0
8	254	256	8	254	255.255.255.0
9	510	512	7	126	255.255.255.128
10	1022	1024	6	62	255.255.255.128
11	2046	2048	5	30	255.255.255.224
12	4094	4096	4	14	255.255.255.240
13	8190	8192	3	6	255.255.255.248
14	16382	16384	2	2	255.255.255.252
15	32766	32768	1	0	255.255.255.254
16	65534	65536	0	0	255.255.255.255

Subredes y máscaras que pueden definirse en una red clase B

Bits de subred	Número de subredes	Nº subredes (subred-cero)	Bits de host	Número de hosts	Máscara
0	0	0	8	254	255.255.255.0
1	0	2	7	126	255.255.255.128
2	2	4	6	62	255.255.255.192
3	6	8	5	30	255.255.255.224
4	14	16	4	14	255.255.255.240
5	30	32	3	6	255.255.255.248
6	62	64	2	2	255.255.255.252
7	126	128	1	0	255.255.255.254
8	254	256	0	0	255.255.255.255

Subredes y máscaras que pueden definirse en una red clase C

Figura 1.2: Subredes y máscaras posibles para redes de tipo B y C

16 subredes de 256 direcciones:

Subred	Máscara	Subred/bits de máscara
156.134.0.0	255.255.255.0	156.134.0.0/24
156.134.1.0	255.255.255.0	156.134.1.0/24
156.134.2.0	255.255.255.0	156.134.2.0/24
156.134.3.0	255.255.255.0	156.134.3.0/24
156.134.4.0	255.255.255.0	156.134.4.0/24
156.134.5.0	255.255.255.0	156.134.5.0/24
Ajustar columna de tabla	255.255.0	156.134.6.0/24
156.134.7.0	255.255.255.0	156.134.7.0/24
156.134.8.0	255.255.255.0	156.134.8.0/24
156.134.9.0	255.255.255.0	156.134.9.0/24
156.134.10.0	255.255.255.0	156.134.10.0/24
156.134.11.0	255.255.255.0	156.134.11.0/24
156.134.12.0	255.255.255.0	156.134.12.0/24
156.134.13.0	255.255.255.0	156.134.13.0/24
156.134.14.0	255.255.255.0	156.134.14.0/24
156.134.15.0	255.255.255.0	156.134.15.0/24

16 subredes de 1024 direcciones:

Subred	Máscara	Subred/bits de máscara
156.134.16.0	255.255.252.0	156.134.16.0/22
156.134.20.0	255.255.252.0	156.134.20.0/22
156.134.24.0	255.255.252.0	156.134.24.0/22
156.134.28.0	255.255.252.0	156.134.28.0/22
156.134.32.0	255.255.252.0	156.134.32.0/22
156.134.36.0	255.255.252.0	156.134.36.0/22
156.134.40.0	255.255.252.0	156.134.40.0/22
156.134.44.0	255.255.252.0	156.134.44.0/22
156.134.48.0	255.255.252.0	156.134.48.0/22
156.134.52.0	255.255.252.0	156.134.52.0/22
156.134.56.0	255.255.252.0	156.134.56.0/22
156.134.60.0	255.255.252.0	156.134.60.0/22
156.134.64.0	255.255.252.0	156.134.64.0/22
156.134.68.0	255.255.252.0	156.134.68.0/22
156.134.72.0	255.255.252.0	156.134.72.0/22
156.134.76.0	255.255.252.0	156.134.76.0/22

3 subredes de 4096 direcciones:

Subred	Máscara	Subred/bits de máscara
156.134.80.0	255.255.240.0	156.134.80.0/20
156.134.96.0	255.255.240.0	156.134.96.0/20
156.134.112.0	255.255.240.0	156.134.112.0/20

Figura 1.3: Ejemplos de subredes y máscaras posibles

1.2.2 Superredes: Classless InterDomain Routing (CIDR)

El rápido crecimiento de la Internet está creando varios problemas, el más importante de los cuales es el agotamiento del espacio de direcciones IP. La causa de esto ha sido en parte la excesiva disparidad de tamaños entre las diferentes clases de redes. Hace ya mucho tiempo que han dejado de asignarse redes clase A, debido a su escaso número y tamaño excesivo. Las organizaciones tenían pues que elegir entre solicitar una clase B o una clase C. En muchos casos una clase B era excesiva, pero una C resultaba insuficiente, por lo que la mayoría de las organizaciones optaban por solicitar una clase B, aunque a menudo no necesitaban tantas direcciones. A la vista del rápido agotamiento de redes clase B debido a este motivo se pensó en crear grupos de clases C, de forma que las organizaciones pudieran optar por niveles intermedios entre las redes B y C, más adecuados a sus necesidades; por ejemplo una organización que necesite 2048 direcciones puede hoy en día solicitar un grupo de ocho redes clase C. De esta forma se reduce el problema de escasez de direcciones, pero se crea un problema nuevo: el crecimiento de las tablas de rutas. Antes, cuando a una organización que se conectaba a Internet se le asignaba una red esto suponía una nueva entrada en las tablas de rutas de Internet, pero al dar grupos de clases C se requiere una entrada diferente para cada red asignada. Esto habría provocado un crecimiento exponencial en las tablas de rutas de los routers que forman el "backbone", cuyas capacidades se encuentran ya bastante cerca del límite de la tecnología.

El gran tamaño de las tablas de rutas de Internet se debe también al mecanismo de asignación de direcciones que se ha seguido, que durante mucho tiempo ha sido estrictamente cronológico. Al no haber una correspondencia entre la ubicación geográfica de una organización o el proveedor que la conecta a Internet y su rango de direcciones no es posible resumir o ?sumarizar? las tablas de rutas; la información se ha de incluir enumerando una a una todas las redes existentes. Esto se podría resolver con una organización jerárquica de direcciones de acuerdo con criterios geográficos, como ocurre por ejemplo en el direccionamiento de la red telefónica que identifica a cada país con prefijo, cada región dentro de un país con un subprefijo, y así sucesivamente.

Actualmente hay mas de 100.000 redes registradas en la Internet. Además del costo en memoria RAM que supone el mantener tablas extremadamente grandes en los routers, los algoritmos de búsqueda se complican y no funcionan adecuadamente con esas tablas, ya que fueron diseñados pensando en tablas con muchas menos entradas. A principios de los 90 el crecimiento de la Internet se estaba produciendo a un ritmo tal que el número de redes conectadas se duplicaba cada 9 meses, mientras que la tecnología sólo permite duplicar la capacidad y potencia de los routers cada 18 meses. En esta situación la explosión de las tablas de routing se estaba convirtiendo en un problema aún más grave que la escasez de direcciones. Según cálculos hechos por la IETF en 1993 de seguir produciéndose el crecimiento al mismo ritmo en el número de redes y rutas la Internet se habría colapsado hacia 1998.

Los dos problemas antes descritos, desperdicio del espacio de direcciones debido a la rigidez

en la asignación de rangos (redes clase B o C) y crecimiento de las tablas de rutas, se resolvieron conjuntamente en 1993 con la adopción de un sistema denominado CIDR (Classless InterDomain Routing) descrito en los RFCs 1466, 1518 y 1519 que consiste en dos medidas complementarias.

La primera medida consiste en establecer una jerarquía en la asignación de direcciones. En vez de utilizar un criterio puramente cronológico, que desde el punto de vista geográfico o de topología de la red equivale a una asignación aleatoria, los rangos se asignan por continentes. Inicialmente se realizó la asignación de una parte del espacio de clase C de la siguiente manera:

Multi regional:	192.0.0.0 - 193.255.255.255
Europa:	194.0.0.0 - 195.255.255.255
Otros:	196.0.0.0 - 197.255.255.255
Norteamérica:	198.0.0.0 - 199.255.255.255
Centro y Sudamérica:	200.0.0.0 - 201.255.255.255
Anillo Pacífico:	202.0.0.0 - 203.255.255.255
Otros:	204.0.0.0 - 205.255.255.255
Otros:	206.0.0.0 - 207.255.255.255

1.3 Protocolos de control de Internet

Todo el tráfico de Internet está formado por datagramas IP. Normalmente los datagramas transportan TPDU's (Transport Protocol Data Unit) de TCP o UDP, que son los dos protocolos de transporte utilizados en Internet. Todas las aplicaciones de interés para el usuario de Internet (correo electrónico, transferencia de ficheros, videoconferencia, etc.) generan tráfico TCP o UDP. Sin embargo cuando vimos en la estructura de la cabecera de un datagrama el valor del campo protocolo observamos que existen muchos posibles significados del contenido de un datagrama IP, aparte de los "normales" que serían TCP y UDP. Algunos de los datos que pueden transportarse en datagramas IP son mensajes de protocolos de control de Internet como los que veremos en este apartado. Los protocolos de control son una parte necesaria para el correcto funcionamiento de la red. Aquí describiremos los más utilizados, que son ICMP, ARP, RARP, BOOTP y DHCP.

1.3.1 ICMP (Internet Control Message Protocol)

En el funcionamiento normal de una red se dan a veces situaciones extraordinarias que requieren enviar avisos especiales. Por ejemplo, si un datagrama con el bit DF (Don't Fragment) puesto a 1 no puede pasar por una determinada red el router donde se produce el problema debe devolver un mensaje al host emisor indicándole lo sucedido. El mecanismo para reportar todos estos incidentes en Internet es el protocolo conocido como ICMP, que veremos a continuación. Aquí solo describiremos

someramente los más importantes, para una descripción detallada de todos ellos se puede consultar el RFC 792.

Conviene recordar que los mensajes ICMP viajan por la red como datagramas IP (con el valor 1 en el campo ?protocolo?), sujetos en los routers a las mismas reglas que cualquier otro datagrama. Los mensajes ICMP son generados por el host o router que detecta el problema o situación extraordinaria y dirigidos al host o router que aparece en el campo dirección origen del datagrama que causó el problema. Para facilitar la identificación del datagrama por parte del host emisor la mayoría de los mensajes ICMP incluyen, además del código de error correspondiente, la cabecera y los primeros ocho bytes de datos del datagrama original.

A continuación describiremos los mensajes ICMP más importantes:

DESTINATION UNREACHABLE: Este mensaje se produce cuando no se puede entregar el datagrama en su destino por diversas situaciones, entre las que destacaremos a modo de ejemplo las dos siguientes: a) cuando un router se encuentra con un datagrama que tiene puesto a 1 el bit DF (Don't Fragment) y que no cabe en la MTU de la red por la que ha de enviarlo, y b) cuando un router no encuentra en sus tablas ninguna ruta por la que pueda llegar a la dirección para la que va dirigido un datagrama.

ECHO REQUEST y ECHO REPLY: se usan para detectar si un destino determinado está operativo. Al recibir el mensaje el destino debe responder con el comando ICMP ECHO REPLY. El comando ping, muy utilizado en Internet para pruebas de accesibilidad, utiliza este comando. En la mayoría de las implementaciones de ping es posible especificar el tamaño del datagrama a enviar, y también la frecuencia de los envíos, con lo que se puede utilizar para generar un tráfico en la red de forma controlada. El comando ping suministra información del tiempo de ida y vuelta y porcentaje de datagramas perdidos, por lo que permite tener una idea bastante aproximada del estado de la red.

TIME EXCEEDED se envía al emisor cuando un paquete es descartado porque su TTL ha llegado a cero. Esto puede ser síntoma de que se ha producido algún bucle en la red, o que el valor del TTL utilizado es demasiado bajo para el diámetro de la red. Hay un programa muy utilizado para diagnóstico de redes, denominado traceroute1, que averigua la ruta a un destino determinado enviando paquetes con TTL de 1, 2, 3, y así sucesivamente. A partir de los mensajes ICMP TIME EXCEEDED recibidos el programa puede deducir la ruta completa hasta el destino especificado. Traceroute mide además el tiempo de ida y vuelta en cada caso, y para dar una información estadísticamente más significativa normalmente envía tres paquetes para cada valor de TTL y muestra los tiempos de ida y vuelta de cada uno, dando así información no sólo de la ruta seguida sino de los retardos en cada parte del trayecto.

REDIRECT se utiliza para alertar al host emisor cuando se sospecha que un paquete se está encaminando incorrectamente. Por ejemplo este mensaje lo utilizan los routers cuando reciben

de un host datagramas que van dirigidos a otro host que se encuentra en la misma LAN.

1.3.2 ARP (Address Resolution Protocol)

Cuando se utilizan líneas punto a punto en una red la interfaz física fija el siguiente nodo al que se dirige el datagrama, ya que para cada enlace existe un único destinatario posible. En cambio cuando se utiliza una red multiacceso (por ejemplo RDSI, ATM o una LAN cualquiera) la tecnología utilizada para enviar los datagramas permite llegar por la misma interfaz física a más de un destinatario. En este caso es necesario algún mecanismo que permita saber a cual de todos los destinos posibles se dirigen los paquetes. Todas las redes multiacceso disponen de un sistema de direccionamiento propio, por ejemplo en una RDSI las direcciones serían los números de teléfono RDSI con los que queremos conectar, en una LAN serían las direcciones MAC de las estaciones, etc. En todos estos casos es el nivel de red el encargado de realizar la correspondencia entre la dirección en la tecnología multiacceso correspondiente y la dirección de red, correspondencia que se conoce como **resolución de direcciones**.

Existen múltiples mecanismos posibles para realizar la resolución de direcciones. Uno de ellos consiste en utilizar un mensaje broadcast para lanzar a la red una pregunta solicitando respuesta del nodo en la red multiacceso que posee la dirección de red buscada. Sin embargo solo es factible en redes de naturaleza broadcast, como las redes locales. El principal inconveniente de esta solución es el uso de paquetes broadcast que produce una degradación del rendimiento de toda la red¹. Esta técnica es la utilizada en IP sobre redes locales de todo tipo.

Veamos como funciona con un ejemplo:

Supongamos que Luis tiene su PC conectado a una red local Ethernet mediante una tarjeta configurada con la dirección IP 147.156.1.2 y la máscara 255.255.0.0; Luis quiere iniciar una sesión de terminal remoto telnet con un ordenador cuya dirección IP es 147.156.1.3. En el momento en que Luis teclea telnet 147.156.1.3 el software del PC compara la dirección de destino con la suya propia y con su máscara y deduce que el ordenador de destino se encuentra en la misma red local. El PC genera entonces un mensaje ARP con la pregunta ¿quién tiene la dirección 147.156.1.3? y lo envía en una trama Ethernet que tiene como dirección MAC de destino la dirección broadcast (todo a unos); la trama es recibida y procesada por todas las máquinas de la red que en ese momento estén activas, siendo retransmitida a través de los conmutadores o puentes transparentes locales o remotos que haya. Eventualmente una máquina (y solo una) se reconoce propietaria de la dirección IP solicitada y responde al mensaje; la respuesta puede ser y normalmente será una trama unicast puesto que el ordenador de destino ya conoce la dirección MAC del PC que lanzó la pregunta; la respuesta incluye la dirección MAC solicitada, por lo que a partir de ese momento ambos ordenadores pueden comunicarse mediante tramas unicast, de forma que la generación de tráfico broadcast queda estrictamente limitada al primer mensaje enviado por el PC de Luis.

Cada ordenador de la red local mantiene en memoria una tabla denominada ARP cache con las parejas de direcciones MAC-IP utilizadas recientemente. Por tanto si Luis termina su sesión telnet pero olvidó algún detalle podrá durante varios minutos establecer otra conexión telnet con el mismo ordenador sin necesidad de enviar otro paquete ARP broadcast. Generalmente cuando un ordenador envía un mensaje ARP buscando a otro todas las máquinas de la red, no sólo la destinataria del mensaje, aprovechan para 'fichar' al emisor, anotándolo en su ARP cache. De esta forma si más tarde necesitan contactar con dicha máquina podrán hacerlo directamente, sin necesidad de enviar un mensaje ARP broadcast.

1.3.3 RARP, BOOTP y DHCP

A veces se plantea el problema inverso a ARP, es decir hallar la dirección IP a partir de una determinada dirección LAN. Por ejemplo cuando se arranca una estación de trabajo 'diskless', es decir sin disco, ésta ha de cargar su sistema operativo desde otro ordenador normalmente situado en la misma red local, pero desconoce todo lo relativo a su configuración de red, incluida la dirección IP; lo único que la estación conoce en principio es su dirección MAC, que se encuentra escrita en su tarjeta de red local.

Para estas situaciones se inventó **RARP (Reverse Address Resolution Protocol)**, que funciona de la siguiente manera: la estación diskless envía un mensaje broadcast en el que indica su dirección MAC y solicita que alguien le informe de cual es la dirección IP que le corresponde. En la red habrá un servidor RARP encargado de atender este tipo de peticiones que consultará sus tablas y devolverá la dirección IP correspondiente.

RARP aporta funcionalidades interesantes pero tiene algunas limitaciones importantes. Como la consulta se realiza mediante un mensaje broadcast el servidor RARP ha de estar en la misma LAN que el cliente; por tanto es necesario prever un servidor RARP en cada LAN, ya que los mensajes broadcast a nivel MAC no atraviesan los routers. Otra limitación es el hecho de que el protocolo RARP solo preve el envío de la dirección IP, cuando sería interesante aprovechar el mensaje para informar al cliente de todo el conjunto de parámetros relacionados con la configuración de la red (la máscara, el router por defecto, etc.). Para superar estas limitaciones de RARP se inventó el protocolo **BOOTP (BOOTstrap Protocol)**.

Cuando un host lanza una pregunta BOOTP lo hace en un datagrama IP con la dirección de destino 255.255.255.255 y dirección de origen 0.0.0.0. De esta forma el datagrama es recibido por todos los hosts de la LAN. Si alguno de ellos es el servidor BOOTP consultará sus tablas y devolverá la información requerida al solicitante, en otro datagrama IP.

Tanto RARP como BOOTP requieren una asignación estática biunívoca entre direcciones MAC y direcciones IP; hacen falta tantas direcciones IP como ordenadores vayan a utilizar el

protocolo TCP/IP. Existen situaciones en las que esto no es conveniente, por ejemplo:

- Una empresa con 500 ordenadores quiere conectarse a la Internet, de forma que cualquiera de ellos pueda salir al exterior; para esto dispone de una clase C; se calcula que nunca habrá más de 200 ordenadores simultáneamente conectados a la Internet, por lo que en principio una red clase C sería suficiente, pero la necesidad de asignar una dirección IP a cada ordenador requiere disponer de 500 direcciones IP si se quiere ofrecer conectividad a todos ellos.
- En una universidad se dispone de una sala para la conexión a Internet de los ordenadores portátiles de los alumnos. No se conocen las direcciones MAC de los ordenadores que se utilizarán ni cuantos serán, lo único que se sabe es que no habrá en ningún momento más de 50 conectados simultáneamente ya que esta es la capacidad de la sala.

Evidentemente en estas situaciones es necesario un mecanismo más flexible de asignación de direcciones IP que el ofrecido por BOOTP.

Para resolver estos problemas el IETF diseñó en 1993 el protocolo **DHCP (Dynamic Host Configuration Protocol)**, que es similar a BOOTP pero es más versátil en los mecanismos de asignación de direcciones IP. En DHCP los clientes pueden recibir sus direcciones por uno de los tres mecanismos siguientes:

- Asignación indefinida y estática. En este caso la dirección es fijada por el administrador. Este equivale a BOOTP.
- Asignación automática. La asignación es también estática pero la elección de la dirección IP correspondiente es tomada por el servidor DHCP la primera vez que el equipo le solicita su dirección.
- Asignación dinámica. En este caso el cliente recibe la dirección IP del servidor durante un tiempo limitado. Pasado ese tiempo el cliente debe renovar su solicitud o de lo contrario la concesión expirará. De esta forma una misma dirección puede ser reutilizada por diferentes máquinas en momentos diferentes. Esta modalidad es también conocida como "alquiler de direcciones".

La mayor flexibilidad de DHCP le ha convertido en el protocolo preferido para la configuración remota de ordenadores en redes locales. Con DHCP se mejora notablemente la seguridad y fiabilidad de una red; también se simplifican las labores de administración de la misma.

Con la asignación dinámica de direcciones DHCP desempeña en redes locales un papel similar a PPP en las conexiones de RTC.

Un inconveniente de la asignación dinámica de direcciones es que si se desea rastrear un problema y, como es lo normal, sólo se dispone de la dirección IP resulta más difícil (a veces imposible) averiguar que ordenador o usuario ha sido el causante del problema. Otro problema es la asociación de direcciones y nombres en el DNS; con la asignación dinámica diferentes máquinas pueden recibir el mismo nombre en diferentes momentos.

2 EL NIVEL DE TRANSPORTE

El nivel de transporte se encarga de suministrar el servicio de transporte de bits a las aplicaciones. Éstas funcionan generalmente según el paradigma cliente-servidor, por el cual una aplicación (cliente) toma la iniciativa y solicita los servicios a la otra (servidor).

Como ya sabemos la comunicación 'peer to peer' entre dos entidades del nivel de transporte ocurre en realidad gracias a los servicios ofrecidos por el nivel de red. Mientras que el nivel de red se ocupa de resolver los problemas propios de la topología de ésta (rutas y congestión fundamentalmente) el nivel de transporte sólo existe en las dos entidades extremas de la comunicación, por lo que también se le llama nivel host-host o extremo a extremo. El nivel de transporte no es consciente, ni debe serlo, de la manera como físicamente están interconectados los dos hosts, que puede ser por una LAN, una WAN o una combinación de múltiples redes de ambos tipos.

La unidad básica de intercambio de información a nivel de enlace se denomina trama (porque los datos van 'rodeados' de información de control por delante y por detrás). En el nivel de red esta unidad básica se conoce como paquete. No existe un término equivalente para la unidad de transferencia de información en el nivel de transporte; a falta de mejor alternativa utilizaremos para este fin el término OSI TPDU (Transport Protocol Data Unit); en la Internet se suele utilizar el término mensaje en el caso de UDP (servicio no orientado a conexión), y segmento en el de TCP (servicio orientado a conexión), pero esta nomenclatura no es compartida por otros protocolos de transporte.

Generalmente las aplicaciones requieren que el nivel de transporte les garantice la entrega de los datos al destinatario, sin errores, pérdidas ni datos duplicados; para que esto sea posible el nivel de transporte ofrecerá normalmente un servicio orientado a conexión, con retransmisiones en caso necesario. Este es el caso por ejemplo del protocolo TCP de Internet, utilizado en muchas aplicaciones como FTP (File Transfer Protocol, transferencia de ficheros), SMTP (Simple Mail Transfer Protocol, correo electrónico), HTTP (HyperText Transfer Protocol, usado en tráfico Web), etc.

En ocasiones las aplicaciones se conforman -o incluso prefieren- un servicio menos fiable

en el que los mensajes se envían sin pedir confirmación, de forma independiente unos de otros. Este tipo de servicio se suministra normalmente con un protocolo no orientado a conexión. El protocolo UDP de Internet es un ejemplo de este tipo de servicio. Entre los casos en que se quiere un servicio de este tipo se encuentran por ejemplo las aplicaciones en tiempo real ya que en ese caso no se quiere incurrir en el retardo propio de un protocolo orientado a conexión.

Normalmente las entidades del nivel de transporte se implementan como procesos en el sistema operativo del host, o bien procesos de usuario. El sistema operativo puede ser monousuario o multiusuario. En muchos casos el host tiene una sola instancia del nivel de red, y una sola del de transporte, pero muchas de los niveles superiores (aplicación o sesión); el nivel de transporte se encarga de multiplexar el tráfico recibido de las diversas entidades de nivel superior en una única conexión a través del nivel de red.

Los protocolos de transporte no orientados a conexión, como UDP, son protocolos muy sencillos, que existen únicamente para permitir la correcta conexión de la capa de aplicación y la de red; actúan como una capa de adaptación bastante primitiva. Por esto la mayoría de nuestra discusión se centrará en los protocolos orientados a conexión, a los que nos referiremos implícitamente la mayor parte del tiempo.

2.1 Primitivas del servicio de transporte

En un servicio de transporte básico orientado a conexión (CONS) la secuencia de primitivas podría ser algo como la que se muestra en la figura 2.1.

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

Figura 2.1: *Primitivas del nivel de transporte*

Para ver cómo se usan estas primitivas, consideremos una aplicación con un servidor y varios clientes remotos. Para comenzar el servicio, el servidor ejecuta la primitiva LISTEN, y esta a su vez realiza una llamada al sistema operativo que bloquea al servidor hasta que aparezca alguna petición de algún cliente. Por su parte, cuando algún cliente desea conectarse con el servidor ejecuta una primitiva CONNECT. La capa de transporte al ejecutar esta primitiva bloquea al proceso cliente y envía un paquete al servidor, en este caso una TPDU "CONNECTION REQUEST". Al llegar esta

al servidor, la capa de transporte del servidor verifica que el servidor está bloqueado en LISTEN. A continuación desbloquea al servidor y envía una TPDU "CONNECTION ACCEPTED" de vuelta al cliente. Al llegar esta solicitud, el cliente se desbloquea y establece la conexión.

En este estado, cliente y servidor pueden intercambiarse TPDUS usando las primitivas SEND y RECEIVE. La RECEIVE es bloqueante, para esperar que la otra parte emita un SEND. Este esquema funciona bien siempre y cuando ambos lados se pongan de acuerdo en quién tiene el turno para transmitir.

Cuando ya no se necesita una conexión, debe liberarse. La conexión puede ser *simétrica* o *asimétrica*. En la variante asimétrica, cualquiera de las 2 entidades de transporte emite una DISCONNECT a la entidad remota, y a la llegada de esta TPDU DISCONNECT se libera la conexión. En la variante simétrica, cada conexión se cierra por separado, independientemente de la otra. Cuando una parte emite un DISCONNECT quiere decir que ya no tiene más datos para enviar, pero aún puede recibir datos de la otra parte. En esta variante, se necesita un DISCONNECT de cada lado para cerrar la conexión.

2.2 La interfaz sockets

La forma en la que se realiza la comunicación vertical entre 2 niveles adyacentes de la arquitectura de protocolos en un mismo computador (es decir, lo que ocurre dentro de un sistema) es asunto interno que no incumbe a los protocolos. Esa comunicación vertical se realiza normalmente mediante las denominadas APIs (Application Programming Interfaces).

Aunque no requerida por las comunicaciones, la estandarización de las APIs comporta unos beneficios evidentes por la posibilidad de aprovechar software entre sistemas diferentes. Esto es especialmente cierto en el nivel de transporte, ya que es aquí donde interaccionarán más programas diferentes, correspondientes a las diversas aplicaciones que se desarrollen. Muchas implementaciones de TCP/IP disponen de una API para programación de aplicaciones denominada sockets (literalmente enchufes, aunque nunca se utiliza esta denominación). Los sockets se introdujeron con el sistema operativo UNIX BSD (Berkeley Software Distribution) en 1982. La interfaz sockets es multiprotocolo, soporta TCP, UDP y otros protocolos.

Aun cuando no forman parte de ningún estándar oficial ni están recogidos en ningún RFC, los sockets son la API más extendida en programación de aplicaciones TCP/IP y forman un estándar 'de facto'. Existen implementaciones para muchos sistemas operativos no UNIX, e incluso en casos en que el sistema operativo no los incorpora suele haber una librería de rutinas sockets que permite adaptar programas con relativa facilidad. Por ejemplo, la interfaz WinSock permite adaptar aplicaciones MS Windows sobre diversas implementaciones de TCP/IP. Al no ser un estándar puede haber pequeñas diferencias entre implementaciones, por lo que es conveniente disponer siempre de

la documentación correspondiente al software que se utiliza.

La filosofía básica de los sockets deriva directamente del sistema de entrada/salida de UNIX, con ampliaciones que permiten por ejemplo a un proceso servidor ponerse 'a la escucha'. Algunas de las rutinas generan y envían las TPDU's a partir de sus argumentos; éstas (las TPDU's) sí forman parte del protocolo, por lo que deben de ser conformes con el estándar correspondiente.

La lista de primitivas *socket* se lista en la figura 2.2

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Attach a local address to a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Block the caller until a connection attempt arrives
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

Figura 2.2: Primitivas de socket para TCP

2.3 Los protocolos de transporte de la Internet: TCP y UDP

Como ya hemos comentado existen dos protocolos de transporte en la Internet: TCP es fiable, orientado a conexión con control de flujo, y UDP es no fiable (sin confirmación) no orientado a conexión y sin control de flujo. La TPDU de TCP se denomina segmento, y la de UDP mensaje o también datagrama UDP.

TCP prevé una comunicación full dúplex punto a punto entre dos hosts, no hay soporte para tráfico multicast. En UDP la comunicación es simplex (aunque obviamente un datagrama UDP puede ser respondido por el receptor con otro); en UDP es posible el tráfico multicast o broadcast.

El protocolo TCP es mucho más complejo que UDP.

2.3.1 TCP (Transport Control Protocol)

Recordemos que el nivel de transporte debe ofrecer algún mecanismo que permita distinguir a que aplicación van dirigidos los datos. En TCP este mecanismo se denomina **ports o puertos**.

En sentido estricto una conexión entre dos entidades usuarias del nivel de transporte queda identificada por los puertos en los que conectan cada una (podemos pensar en el puerto como el conector telefónico, diríamos entonces que una conversación telefónica queda perfectamente especificada por los números de teléfono de las dos rosetas donde están enchufados los aparatos con los que se está hablando). Recordaremos que un puerto en Internet está especificado por:

- Dirección donde "conecta" el nivel de red: dirección IP de los dos hosts
- Dirección donde conecta el nivel de transporte (campo protocolo del datagrama IP): normalmente TCP, ya que UDP al ser no orientado a conexión no puede establecer conexiones.
- Dirección donde conecta el nivel de aplicación: esto es el puerto.

Dado que la conexión en el nivel de transporte siempre se suele realizar con el protocolo TCP, este dato es innecesario y se suele omitir. Sin embargo en un mismo host un número de port puede ser utilizado simultáneamente por una aplicación para UDP y por otra para TCP; esto no plantea ningún conflicto ya que son puertos diferentes.

Así pues, una conexión de dos entidades usuarias del nivel de transporte se especifica por la combinación:

Dirección IP host 1 + port host 1 + dirección IP host 2 + port host 2

El port es un número entero entre 0 y 65535. Por convenio, los números 0 a 1023 están reservados para el uso de servicios estándar, por lo que se les denomina puertos bien conocidos o *well-known* ports. Cualquier número por encima de 1023 está disponible para ser utilizado libremente por los usuarios. Los valores vigentes de los puertos bien conocidos se pueden consultar por ejemplo en el web de la IANA (Internet Assigned Number Authority) www.iana.org/numbers.html. En la figura 2.3 se recogen algunos de los más habituales.

Para comprender la relación entre los puertos y las conexiones en TCP veamos un ejemplo concreto: supongamos que cinco usuarios desde un host de dirección IP 134.123.1.2 inician una sesión de logon remoto (Telnet) hacia el host de dirección 221.198.34.21; cada uno de ellos ejecutará en su host un programa telnet cliente que abrirá una conexión con el servidor Telnet (puerto 23) en el otro; las conexiones establecidas podrían ser por ejemplo:

Usuario 1: 134.123.1.2.1024 con 221.198.34.21.23

Usuario 2: 134.123.1.2.1025 con 221.198.34.21.23

Usuario 3: 134.123.1.2.1026 con 221.198.34.21.23

Port	Protocol	Use
21	FTP	File transfer
23	Telnet	Remote login
25	SMTP	E-mail
69	TFTP	Trivial file transfer protocol
79	Finger	Lookup information about a user
80	HTTP	World Wide Web
110	POP-3	Remote e-mail access
119	NNTP	USENET news

Figura 2.3: Ejemplos de puertos bien conocidos en TCP

Usuario 4: 134.123.1.2.1030 con 221.198.34.21.23

Usuario 5: 134.123.1.2.1031 con 221.198.34.21.23

Aquí hemos empleado la notación *dirección IP. Puerto* para identificar el socket1; cada conexión queda identificada de forma no ambigua por los dos sockets que conecta. Obsérvese que la asignación de puertos para los clientes se hace por simple orden de llegada a partir del primer número de puerto no reservado. En el servidor todas las conexiones utilizan el puerto 23 (pues todas acceden al mismo proceso, el servidor telnet); en cambio en el cliente cada usuario es un proceso diferente y utiliza un puerto distinto.

Complicando un poco más el ejemplo anterior podríamos imaginar que el host cliente estuviera 'multihomed', es decir que tuviera dos interfaces físicas (por ejemplo dos tarjetas LAN) y por tanto tuviera dos direcciones IP; supongamos que los usuarios utilizan ambas interfaces alternativamente, en ese caso las conexiones podrían ser:

Usuario 1: 134.123.1.2.1024 con 221.198.34.21.23

Usuario 2: 134.123.1.3.1024 con 221.198.34.21.23

Usuario 3: 134.123.1.2.1025 con 221.198.34.21.23

Usuario 4: 134.123.1.3.1025 con 221.198.34.21.23

Usuario 5: 134.123.1.2.1030 con 221.198.34.21.23

Por otro lado el host cliente podría simultáneamente a las sesiones telnet enviar datagramas UDP al servidor. Aunque en este caso no se establece una conexión (pues se trata de un servicio CLNS) hay un puerto de origen y uno de destino; podría haber datagramas que tuvieran como puerto de origen el 1024 en el host 134.123.1.2 y como destino el 23 en 221.198.34.21; esto no causaría ninguna ambigüedad ya que el campo protocolo de la cabecera IP permitiría distinguir ambos tipos de paquetes entregando cada uno al servicio correspondiente del nivel de transporte en el host de destino.

2.3.2 La cabecera de segmento TCP

La cabecera de un segmento TCP tiene la estructura que se muestra en la figura 2.4

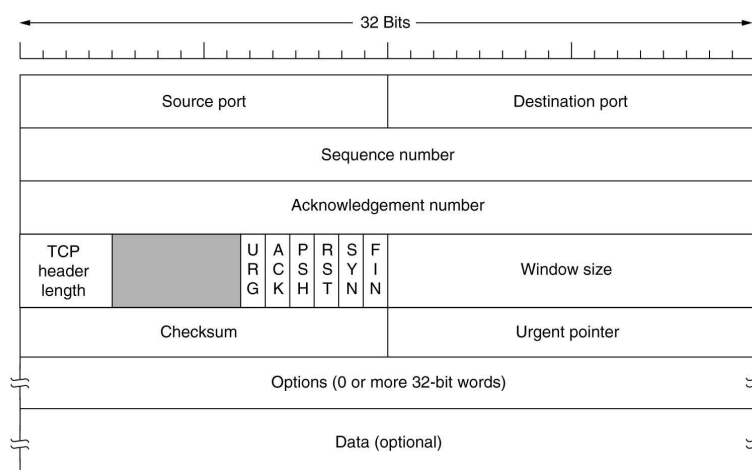


Figura 2.4: Cabecera de un segmento TCP

Puerto origen y puerto destino: identifican los puertos que se van a utilizar en cada host para comunicar con las aplicaciones que intercambian datos.

Número de secuencia: indica el número de secuencia que corresponde en la conexión al primer byte que se envía en el campo datos de ese segmento.

Número de ACK: indica el número de secuencia del primer byte del próximo segmento que se espera recibir del otro lado.

Longitud de cabecera TCP: especifica la longitud en palabras de 32 bits, excluido el campo datos (el campo opciones hace que dicha longitud pueda variar).

A continuación hay 4 bits no utilizados, seguidos por ocho flags indicadores de un bit de longitud cada uno:

CWR: Congestion Window Reduced. Tiene que ver con el control de congestión de IP que no describiremos aquí

ECE: ECN Echo (ECN=Explicit Congestion Notification). Tiene que ver con el control de congestión de IP que no describiremos aquí.

URG (urgent): sirve para indicar que el segmento contiene datos urgentes; en ese caso el campo puntero de datos urgentes contiene la dirección donde terminan éstos.

ACK (acknowledgement): indica que en este segmento el campo Número de ACK tiene el significado habitual (número del próximo byte que se espera recibir), de lo contrario carece de significado. En la práctica el bit ACK esta a 1 siempre, excepto en el primer segmento enviado por el host que inicia la conexión.

PSH (push): indica que el segmento contiene datos PUSHed. Esto significa que deben ser enviados rápidamente a la aplicación correspondiente, sin esperar a acumular varios segmentos.

RST (reset): se usa para indicar que se debe abortar una conexión porque se ha detectado un error de cualquier tipo; por ejemplo una terminación unilateral de una conexión o que se ha recibido un segmento con un valor inadecuado del número de secuencia o número de ACK, posiblemente producido por un duplicado retrasado de un intento de conexión.

SYN (synchronize): este bit indica que se está estableciendo la conexión y está puesto sólo en el primer segmento enviado por cada uno de los dos hosts en el inicio de la conexión.

FIN (finish): indica que no se tienen más datos que enviar y que se quiere cerrar la conexión. Para que una conexión se cierre de manera normal cada host ha de enviar un segmento con el bit FIN puesto.

Tamaño de ventana: indica la cantidad de bytes que se está dispuesto a aceptar del otro lado en cada momento. Se supone que se garantiza una cantidad suficiente de espacio en buffers. Mediante este parámetro el receptor establece un control de flujo sobre el caudal de datos que puede enviar el emisor.

Checksum: sirve para detectar errores en el segmento recibido; estos podrían ser debidos a errores de transmisión no detectados, a fallos en los equipos (por ejemplo en los routers) o a problemas en el software (por ejemplo reensamblado incorrecto de fragmentos). Recordemos que el datagrama IP contenía un checksum, pero aquel solo comprendía la información de cabecera y además ese checksum desaparece en IPv6. El algoritmo utilizado en TCP es el mismo que el de IP: sumar todos los campos de 16 bits usando aritmética de complemento a 1 y calcular el complemento

a 1 del resultado, pero en este caso el checksum se hace sobre todo el segmento, incluidos los datos, no sólo sobre la información de cabecera. Para el cálculo del checksum se antepone al segmento una pseudocabecera que incluye la dirección IP de origen y destino, el protocolo de transporte utilizado (TCP en este caso) y la longitud del segmento. La pseudocabecera (así denominada porque sólo se utiliza en el cálculo, pero no forma parte del segmento) permite a TCP comprobar que no ha habido errores en la transmisión a nivel IP (detectar por ejemplo cuando un segmento ha sido entregado al host equivocado).

Puntero de datos urgentes: indica el final de éstos, ya que el segmento podría contener datos no urgentes. TCP no marca el principio de los datos urgentes, es responsabilidad de la aplicación averiguarlo.

El campo opciones habilita un mecanismo por el cual es posible incluir extensiones al protocolo. Entre las más interesantes se encuentran las siguientes:

Tamaño máximo de segmento

Uso de repetición selectiva (en vez de retroceso n)

Uso de NAK (acuse de recibo negativo en caso de no recepción de un segmento)

Uso de ventana mayor de 64 Kbytes mediante el empleo de un factor de escala

2.4 UDP (User Datagram Protocol)

TCP tiene la robustez y funcionalidades propias de un protocolo de transporte orientado a conexión; sin embargo esa robustez y funcionalidad conllevan una cierta complejidad; por ejemplo cualquier transmisión de información TCP requiere como mínimo el intercambio de seis mensajes para establecer la comunicación y terminarla; además mientras una conexión existe ocupa una serie de recursos en el host. A veces no se requiere toda esa funcionalidad; en esos casos se prefiere que el nivel de transporte preste un servicio más sencillo, no orientado a conexión y no fiable (por no fiable queremos decir que el receptor no acusa recibo de las TPDUs enviadas). Algunos ejemplos de situaciones en las que es más conveniente un servicio no orientado a conexión son las siguientes:

El tipo de aplicación no requiere una fiabilidad total y no puede tolerar el retardo producido por los ACKs y las retransmisiones de TCP; este es el caso por ejemplo en la transmisión de vídeo o audio en tiempo real.

La aplicación por su propia naturaleza requiere el envío de uno o dos mensajes únicamente; ejemplos de estas aplicaciones son las siguientes: sincronización de relojes (NTP), consultas al servidor de nombres (DNS), mensajes de gestión de la red (SNMP), etc.; en el DNS no se considera

necesario un transporte fiable porque si se pierde el datagrama la aplicación lo reenviará; en el caso de NTP o SNMP la pérdida de un datagrama no es importante porque la información se está actualizando a intervalos regulares (por ejemplo cada 5 minutos).

Se desea hacer envíos multidestino (multicast o broadcast); esto sólo es posible con un protocolo no orientado a conexión, ya que por su propia naturaleza los protocolos orientados a conexión son punto a punto (en TCP no es posible establecer conexiones multipunto).

El protocolo no orientado a conexión de Internet se conoce como UDP (User Datagram Protocol); su nombre ya da una idea de la naturaleza no fiable del servicio de transporte ofrecido. Entre las aplicaciones que utilizan UDP se encuentran TFTP (Trivial File Transfer Protocol), DNS (Domain Name Server), SNMP (Simple Network Management Protocol), NTP (Network Time Protocol) y NFS (Network File System)¹, etc.

Las TPDU's intercambiadas por UDP se denominan mensajes o datagramas UDP. Recordemos que los mensajes UDP se identifican por el valor 17 en el campo protocolo del datagrama IP.

Una característica interesante de UDP es que puede ser utilizado por aplicaciones que necesitan soporte de tráfico multicast o broadcast. Con TCP esto no es posible debido a la naturaleza punto a punto, orientada a conexión del protocolo.

UDP no suministra ningún mecanismo de control de flujo o control de congestión. Cuando lo que se envía es únicamente un mensaje (por ejemplo una consulta al DNS) esto es innecesario, ya que presumiblemente un mensaje aislado no creará problemas de congestión y será siempre aceptado en destino (y de no ser así el mismo problema habría surgido con TCP para el inicio de la conexión). Si se va a enviar un flujo de mensajes, por ejemplo vídeo o audio en tiempo real, se deberán tomar las medidas adecuadas para asegurar la capacidad suficiente en la red (mediante mecanismos de reserva de capacidad, por ejemplo) y evitar la congestión no excediendo lo solicitado en el momento de hacer la reserva; afortunadamente en estos casos se suele conocer a priori con bastante aproximación el tráfico que se va a introducir en la red.

En caso de congestión en la red parte de los datagramas serán descartados por la red sin informar por ningún mecanismo al emisor, ni al receptor. En caso de saturación del receptor este sencillamente ignorará los datagramas que no pueda aceptar. En algunos se contemplan a nivel de aplicación mecanismos de control que permiten al receptor detectar si se producen pérdidas (por ejemplo numerando los datagramas) informando al emisor para que baje el ritmo de emisión si se rebasa un umbral determinado.

De forma similar a los segmentos TCP, los mensajes UDP se dirigen a la aplicación adecuada mediante el puerto de destino, especificado en la cabecera. Análogamente a TCP los puertos UDP se identifican mediante un campo de 16 bits (números entre 0 y 65535). Aun en el caso de coincidir en número con un puerto TCP son TSAPs diferentes, como ya hemos comentado antes. Al igual

que en TCP los valores por debajo de 1024 están reservados para los puertos denominados 'bien conocidos' (well-known ports), aunque su significado es diferente en la mayoría de los casos, pues también lo son los servicios. La tabla 2.1 muestra algunos de los puertos UDP más utilizados.

Servicio	Puerto	Descripción
Echo	7	Devuelve el datagrama al emisor
Discard	9	Descarta el datagrama
Daytime	13	Devuelve la hora del día
Quote	17	Devuelve una "frase del día"
Chargen	19	Generador de caracteres
Nameserver	53	Servidor de nombres de dominios
Bootps	67	Puerto servidor utilizado para cargar información de configuración Bootp
Bootpc	68	
TFTP	69	Trivial File Transfer Protocol
SunRPC	111	Sun Remote Procedure Call
NTP	123	Network Time Protocol
SNMP	161	Usado para recibir consultas de gestión de la red
SNMP-trap	162	Usado para recibir avisos de problemas en la red

Tabla 2.1: Algunos de los puertos UDP más utilizados

La estructura de un mensaje UDP se muestra en la tabla 2.2

Campo	Longitud (bits)
Puerto origen	16
Puerto destino	16
Longitud	16
Checksum	16
Datos	0-524056 (65507 bytes)

Tabla 2.2: Estructura de un mensaje UDP

A continuación describimos el significado de cada uno de los campos de la cabecera UDP:

Puerto origen especifica el puerto de la aplicación que genera el mensaje. Este valdrá normalmente cero, salvo que la aplicación solicite una respuesta.

Puerto destino especifica el puerto de la aplicación a la que va dirigido el mensaje.

Longitud indica la longitud del mensaje, incluyendo los campos de cabecera.

Checksum: el uso de este campo es opcional en IPv4, obligatorio en IPv6 (ya que en ese caso se ha suprimido el checksum a nivel de red). Cuando se envía información en tiempo real (audio o vídeo digitalizado) su uso puede omitirse. Para el cálculo se aplica el mismo algoritmo que

en TCP (suma complemento a 1 de todo el mensaje dividido en campos de 16 bits, y complemento a 1 del resultado). En el cálculo se utiliza todo el mensaje, incluida la cabecera y se antepone una pseudocabecera similar a la utilizada en TCP (con la dirección IP de origen, de destino, el tipo de protocolo de transporte y la longitud del mensaje) de forma que se verifica que sean correctos no solo los datos del mensaje UDP sino también los datos fundamentales de la cabecera IP. Si la verificación del checksum en el receptor da error el mensaje es simplemente descartado sin notificarlo al nivel de aplicación ni al emisor.

Datos contiene los datos a transmitir. Un mensaje UDP ha de estar contenido necesariamente en un datagrama IP, lo cual fija la longitud máxima de este campo.

De la misma forma que un host o un router pueden tener que fragmentar un datagrama que contenga un segmento TCP, es posible que el host emisor o algún router intermedio tengan que fragmentar un mensaje UDP porque sea mayor que la MTU permitida en la red por la que ha de enviarse. Análogamente a los segmentos TCP la fragmentación ocurre de forma transparente a UDP y la cabecera del mensaje solo aparecerá en el primer fragmento; en cambio cada fragmento deberá incluir una nueva cabecera IP.

El funcionamiento del protocolo UDP está descrito en el RFC 768.

Bibliografía

- [1] Andrew S. Tanenbaum, "*Computer Networks*", Ed. Prentice Hall, 3ª ed., 2003.